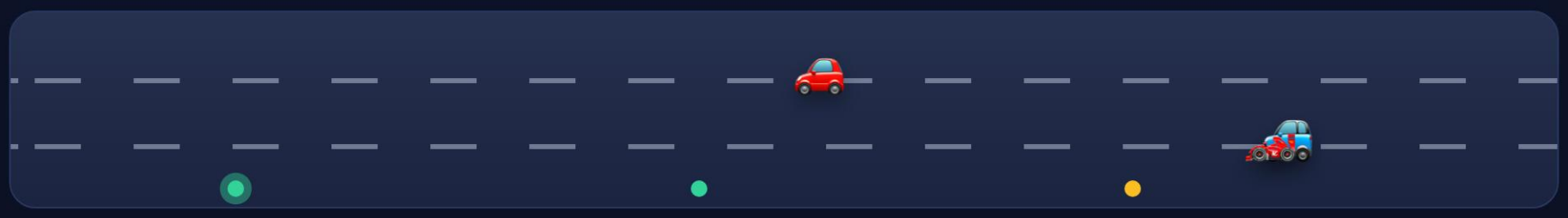
A FRIENDLY TOUR · NO TECH KNOWLEDGE NEEDED

Smart City Traffic Platform

A "brain" for a whole city's roads — it watches traffic, predicts jams before they happen, and re-times traffic lights so everyone moves better. Let's look inside, no jargon. 👇



scroll to explore

↓

💡 In one sentence

It turns a live stream of traffic measurements into predictions, alerts, and smarter traffic-light timings — shown on a real-time map, a phone app, and a talking assistant.

🧠 Think of it like...

...the **nervous system** of a city.

🏙️ Sensors and cameras are the **senses** (they feel how busy each road is). A super-fast "spinal cord" carries those signals. A **brain** decides what's happening and what to do — then warns drivers, suggests detours, and gives busy directions more green light. All in **seconds**, not after you're already stuck. 🚙💨

🔄 The journey of a traffic reading

Follow one little measurement on its trip through the system:

1. Sensed

A road sensor notes "12 cars, 30 km/h, a bit full."

2. Delivered

It rides a fast "conveyor belt" everyone can read.

3. Understood

Models predict the next jam & spot incidents.

4. Acted on

Map updates, drivers get routes, lights re-time.

The glowing dots are little packets of data flowing left to right — exactly how it really moves. 🌊

🧩 The main parts (in plain English)

Hover a card to lift it. Each has a "Coder words" note that decodes the real term.

The data firehose

Thousands of sensor + camera readings a second, never dropped, readable by every part.

> **Coder words:** "Kafka" — an event stream, like a conveyor belt of messages.

The camera watcher

Looks at video, counts cars by type, and flags a stopped car or a wrong-way driver.

> **Coder words:** "computer vision / YOLO" — software that finds objects in images.

The forecaster

Learned from months of past traffic to predict the next 15–60 minutes per road.

> **Coder words:** "machine learning model" — it finds patterns instead of fixed rules.

The light optimizer

Practised in a traffic video-game millions of times to learn timings that cut waiting.

> **Coder words:** "reinforcement learning" — learning by trial, error, and reward.

The living map & 3-D twin

A website showing every road's colour live, plus a 3-D model of the city with moving cars.

> **Coder words:** "frontend + digital twin" — the screen you see, mirroring reality.

The assistant

Ask by voice or text — "Where's it worst?" — and it answers in English, Hebrew, or Arabic.

> **Coder words:** "LLM assistant (Claude) with tools" — an AI that can look things up for you.

The front desk

One secure door for the website & app — checks who you are and what you may see.

> **Coder words:** "API gateway + auth" — the reception desk that checks IDs.

The privacy trick

Neighbourhoods improve the shared AI together — without ever handing over their raw data.

> **Coder words:** "federated learning" — train together, keep your data at home.

👩‍💻 Show me some actual code!

Here's a tiny made-up example — the kind of thing that decides a road's colour. Read the friendly notes below it.

friendly notes below it.

traffic_colour.py

```
# Decide the colour for a road, given its current speed.
def road_colour(speed_kmh, speed_limit):
    ratio = speed_kmh / speed_limit          # how free-flowing is it?
    if ratio > 0.7:
        return "green"                   # moving nicely 🟢
    elif ratio > 0.4:
        return "orange"                  # getting busy 🟠
    return "red"                          # jammed 🔴
```

The grey line with # is a **comment** — a note for humans; the computer ignores it.

def road_colour(...) defines a **function**: a reusable mini-machine you give inputs and it gives an answer.

if / elif is a **decision**: "if this is true, do that; otherwise check the next thing."

return hands the answer back — here, the word "green", "orange", or "red".

📖 Mini-dictionary (6 coder words)

Service — one worker that does one job well (e.g. only handles cameras).

API — a menu of requests one program offers to others. "I'll take a #3, please."

Database — a giant, well-organised filing cabinet the software writes to and reads from.

Container — a lunchbox holding a program plus everything it needs, so it runs anywhere.

Cloud — someone else's powerful computers you rent, instead of buying your own.

Real-time — happening now-ish — updates arrive in seconds, not tomorrow.

📊 By the numbers

The size of the build (the numbers count up when they scroll into view):

6	3	3	12	5
independent services	programming languages	languages on screen (EN · العربية · עברית)	simulated road segments	AI / ML techniques

🖼️ See it (preview)

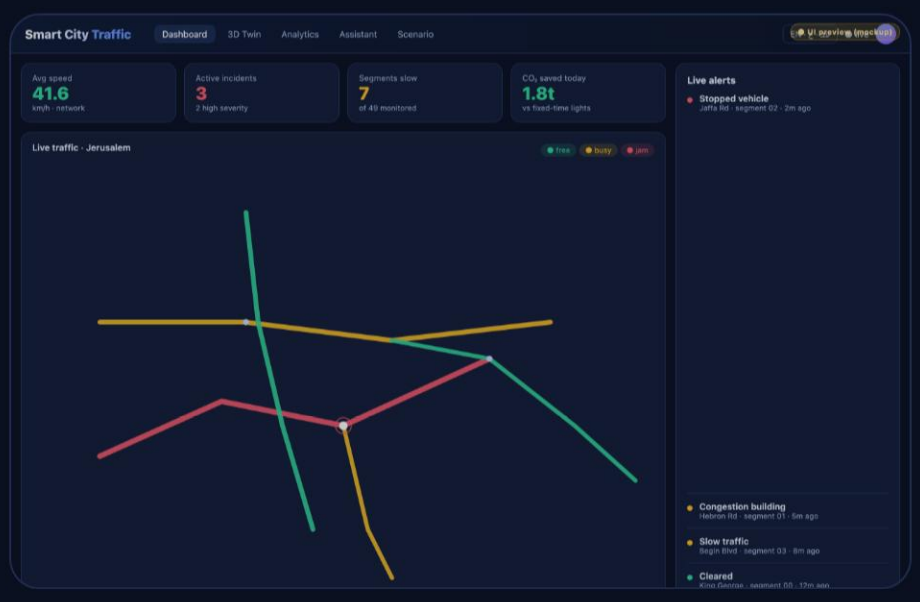
A quick tour of the key screens. These are **UI previews (mockups)** — polished designs of the interface, not photos of the running app (see the honest note below).

Smart City Traffic Platform

a product tour

predicts jams · optimizes lights · 3D twin · AI assistant

UI preview (mockups) — designs, not screenshots of the running app



🤝 One honest note

The cars, sensors, and cameras here are **realistic simulations**, not a real city's live feed — so the whole thing runs safely on a laptop for a demo. Any app screenshots shown elsewhere are labelled **"UI preview (mockup)"** when they're designs rather than photos of the running app. The clever part is the engineering of the system itself. ✅

Built as a portfolio project by **Yousef Hasan Hamda**.

Want the technical version? It's all in the project's README and architecture docs.

📖 Read the full technical story